

1 Crossbar Fixed-Input Analysis

1.0.1 Sneak Current Modeling: Sequential Sensing vs. Full Nodal Analysis

This notebook analyzes a 4×4 resistive crossbar computing $\mathbf{W} \cdot \mathbf{T}$ @ $\mathbf{v}$ for a fixed input $\mathbf{v} = [10, 20, 30, 40]$.

Two simulation approaches are compared:

Model	R_{src}	What it captures
Sequential (original)	—	One column grounded at a time; other columns float. Simple and fast, but misses inter-column coupling.
Nodal	$0 \, \Omega$	All columns sensed simultaneously with ideal row drivers. Eliminates floating-node artifacts; matches sequential when sources are ideal.
Nodal	$10 \, \text{m}\Omega$	Small but finite row resistance. Row node voltages begin to droop under load, introducing measurable error.
Nodal	$100 \, \text{m}\Omega$	Significant row resistance. Voltage droop couples all columns through the shared row network — the dominant real-world sneak current mechanism.

Why it matters: In hardware, row drivers are never ideal. Wire resistance, transistor on-resistance, and parasitic inductance all add finite R_{src} . As R_{src} grows, the row node voltage sags when it sources heavy current, reducing the effective input seen by every column connected to that row. This creates a systematic underestimate of large outputs and asymmetric error across columns — errors that the simple sequential model cannot predict.

```
[1]: import numpy as np

W      = np.array([[1,-1,1,-1],[1,1,-1,-1],[-1,1,1,-1],[-1,-1,-1,1]])
R      = 1.0
R_sense = 0.001

def measure_all_columns(v):
    """Ground each column in turn, return measured currents and floating
    ↪ voltages."""
    v = np.array(v, dtype=float)
    results = {}
```

```

for c in range(4):
    I = (W[:, c] * v).sum() / R / (1 + R_sense * np.abs(W[:, c]).sum() / R)
    floats = {}
    for cf in range(4):
        if cf == c: continue
        SW = W[:, cf].sum()
        SWV = (W[:, cf] * v).sum()
        if abs(SW) > 1e-10: floats[cf] = f"{SWV/SW:+.2f}V"
        elif abs(SWV) < 1e-10: floats[cf] = "0V"
        else: floats[cf] = "RAILS"
    results[c] = {'I': I, 'floats': floats}
return results

```

1.0.2 Section 1 — Sequential Sensing (Original Model)

Each column is grounded individually while the others float. The measured current and floating voltages are shown below. RAILS indicates a floating column whose weight column sums to zero but whose weighted input does not — the node cannot settle to a finite voltage, which is the floating-node hazard SDS is designed to prevent.

```

[2]: v = np.array([10, 20, 30, 40], dtype=float)
i_ideal = W.T @ v
res      = measure_all_columns(v)

print(f"Input: v = {v.astype(int).tolist()}")
print(f"Ideal output (W.T @ v): {i_ideal.astype(int).tolist()}")
print()
print(f"  {'Col':>4}  {'I_sense':>9}  {'I_ideal':>9}  {'Error':>9}  Float nodes_
↳(c#:voltage)")
print(f"  {'-'*60}")
for c in range(4):
    I      = res[c]['I']
    floats = res[c]['floats']
    fstr   = " ".join(f"c{cf}:{fv}" for cf, fv in floats.items())
    print(f"  {c:>4}  {I:>9.4f}  {i_ideal[c]:>9.4f}  {I-i_ideal[c]:>9.6f}  ↳
↳{fstr}")

```

Input: v = [10, 20, 30, 40]

Ideal output (W.T @ v): [-40, 0, -20, -20]

Col	I_sense	I_ideal	Error	Float nodes (c#:voltage)
0	-39.8406	-40.0000	0.159363	c1:0V c2:RAILS c3:+10.00V
1	0.0000	0.0000	0.000000	c0:RAILS c2:RAILS c3:+10.00V
2	-19.9203	-20.0000	0.079681	c0:RAILS c1:0V c3:+10.00V
3	-19.9203	-20.0000	0.079681	c0:RAILS c1:0V c2:RAILS

1.0.3 Section 2 — Full Nodal Analysis with Finite Row Impedance

Solves the complete $(N+M) \times (N+M)$ conductance matrix simultaneously. With $R_{\text{src}} = 0$ the result is identical to the sequential model — confirming no sneak paths exist under ideal conditions. Increasing R_{src} reveals how row voltage droop couples columns and degrades accuracy.

```
[3]: def measure_nodal(v, R_src=0.0):
    """
    Full nodal analysis of the crossbar.
    All columns sensed simultaneously. Optional finite row source resistance,
    ↪ R_src.

    With R_src=0 (ideal): columns decouple, same result as measure_all_columns.
    With R_src>0: row node voltages droop under load, coupling columns together.
    """
    v = np.array(v, dtype=float)
    N, M = W.shape
    g_cell = 1.0 / R
    g_sens = 1.0 / R_sense

    if R_src == 0:
        # Rows clamped to v; columns independent
        V_c = (W.T @ v) / (N + R / R_sense)
        return V_c / R_sense

    g_src = 1.0 / R_src
    # Build (N+M) x (N+M) conductance matrix
    # unknowns: [V_r0..V_r3, V_c0..V_c3]
    A = np.zeros((N + M, N + M))
    b = np.zeros(N + M)

    # Row node k: KCL (v[k]-V_r[k])/R_src = sum_c ↪
    ↪ W[k,c]*(V_r[k]-W[k,c]*V_c[c])/R
    # => (g_src + M*g_cell)*V_r[k] - g_cell * sum_c W[k,c]*V_c[c] = g_src*v[k]
    A[:N, :N] = np.diag(np.full(N, g_src + M * g_cell))
    A[:N, N:] = -g_cell * W
    b[:N] = g_src * v

    # Column node c: KCL sum_k W[k,c]*(V_r[k]-W[k,c]*V_c[c])/R = V_c[c]/R_sense
    # => -g_cell * W.T @ V_r + (N*g_cell + g_sens)*V_c[c] = 0
    A[N:, :N] = -g_cell * W.T
    A[N:, N:] = np.diag(np.full(M, N * g_cell + g_sens))

    X = np.linalg.solve(A, b)
    V_c = X[N:]
    return V_c / R_sense
```

```

# Comparison
v = np.array([10, 20, 30, 40], dtype=float)
i_ideal = W.T @ v

i_orig = np.array([measure_all_columns(v)[c]['I'] for c in range(4)])
i_nod0 = measure_nodal(v, R_src=0.0) # ideal sources, simultaneous
i_nod1 = measure_nodal(v, R_src=0.01) # 10 mΩ row resistance
i_nod2 = measure_nodal(v, R_src=0.1) # 100 mΩ row resistance

print(f"Input v = {v.astype(int).tolist()}")
print(f"Ideal (W.T @ v) = {i_ideal.tolist()}")
print()
print(f"{'Model':<30} {'i0':>8} {'i1':>8} {'i2':>8} {'i3':>8}")
print("-" * 65)
for label, arr in [
    ("sequential (original)", i_orig),
    ("nodal R_src=0 (ideal)", i_nod0),
    ("nodal R_src=0.01", i_nod1),
    ("nodal R_src=0.10", i_nod2),
]:
    vals = " ".join(f"{x:>8.2f}" for x in arr)
    print(f"{'label':<30} {vals}")

print()
print("Error vs ideal:")
print("-" * 65)
for label, arr in [
    ("sequential (original)", i_orig),
    ("nodal R_src=0 (ideal)", i_nod0),
    ("nodal R_src=0.01", i_nod1),
    ("nodal R_src=0.10", i_nod2),
]:
    errs = " ".join(f"{x:>8.4f}" for x in arr - i_ideal)
    print(f"{'label':<30} {errs}")

```

Input v = [10, 20, 30, 40]

Ideal (W.T @ v) = [-40.0, 0.0, -20.0, -20.0]

Model	i0	i1	i2	i3
sequential (original)	-39.84	0.00	-19.92	-19.92
nodal R_src=0 (ideal)	-39.84	0.00	-19.92	-19.92
nodal R_src=0.01	-38.31	0.00	-19.15	-19.15
nodal R_src=0.10	-28.46	0.00	-14.23	-14.23

Error vs ideal:

sequential (original)	0.1594	0.0000	0.0797	0.0797
nodal R_src=0 (ideal)	0.1594	0.0000	0.0797	0.0797
nodal R_src=0.01	1.6906	0.0004	0.8455	0.8462
nodal R_src=0.10	11.5363	0.0020	5.7692	5.7732